

第1次实验 数据编解码电路设计

1.实验环境

- (1)Windows系统下运行Dev-C++（或其他C语言开发环境）。
- (2)Windows系统下运行Logisim软件（需安装JDK）。

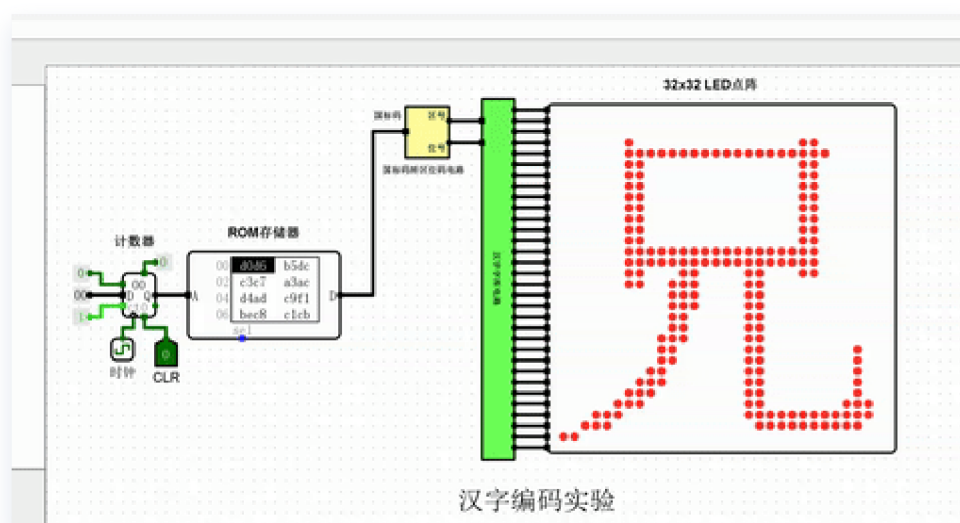
2.实验目的

- (1)运行C语言程序，结合结果进行分析，深度理解数据信息表示中的相应内容；
- (2)通过汉字编码、奇偶校验码、海明码、CRC码等电路验证过程，理解和掌握各种校验码方法的原理与特点；
- (3)在已有电路基础上，进一步完成设计实验与挑战性实验，进行能力提升。

3.1验证性实验

(2)自定义点阵显示文件

实验结果



实验代码

ROM内容如下

```
v2.0 raw
```

```
d0d6 b5dc c3c7 a3ac d4ad c9f1 bec8 c1cb ced2 d2bb c3fc a3ac d4ad c9f1 c5a3 b1c6
```

实验分析

国标码转区位码电路分析

该电路的核心是实现减法逻辑 $\text{区位码} = \text{国标码} - 2020H$ 。

- **输入端**：接收16位国标码。
- **处理逻辑**：国标码的编码规则是在区位码基础上加 `2020H`（即十六进制的 `0x2020`）。为了还原成字库索引所需的区位码，必须进行减法运算。
- **实现细节**：电路中使用了**减法器组件**，将输入的16位国标码减去固定的 `0x2020` 常量。

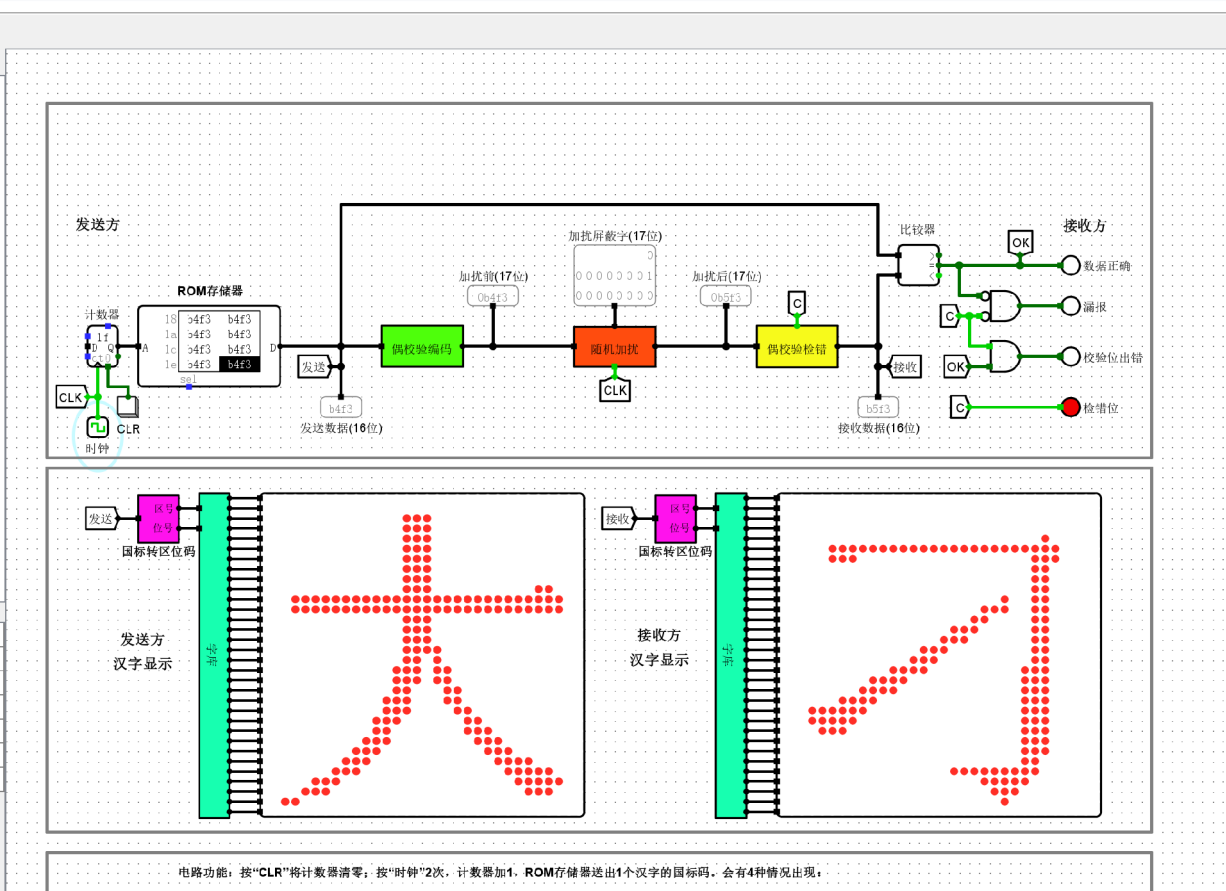
汉字字库电路分析

该电路的核心是一个**ROM组件**，它是字库的载体。

- **地址输入**：输入端连接的是上一级转换电路输出的“区位码”。这个区位码被用作ROM的地址寻址信号，决定了字库读取哪一个汉字的点阵数据。
- **数据存储**：ROM 内部预存了汉字的点阵信息（通常每 32 字节对应一个 16x16 汉字点阵）。电路通过地址寻址，从ROM中按序读出点阵行数据。

(3)16位偶校验传输测试实验

实验结果



实验分析

偶校验编码电路分析

- **电路功能：**在发送端将原始的 16 位数据转换为包含 1 位校验位的 17 位数据。
- **实现原理：**该电路的核心是**级联异或门树**。异或运算具有“奇偶判定”特性：若参与运算的所有数据位中“1”的个数为奇数，则异或输出为 **1**；若“1”的个数为偶数，则输出为 **0**。

在偶校验编码中，若原始 16 位数据中“1”的个数为奇数，异或运算输出 **1**，此时将该输出作为校验位补在末尾，使得总的“1”的个数变为偶数；若原数据中“1”的个数已为偶数，则异或输出 **0**，校验位设为 **0**。通过此电路，实现了将数据编码为符合“偶校验规则”的 17 位信息。

偶校验检错电路分析

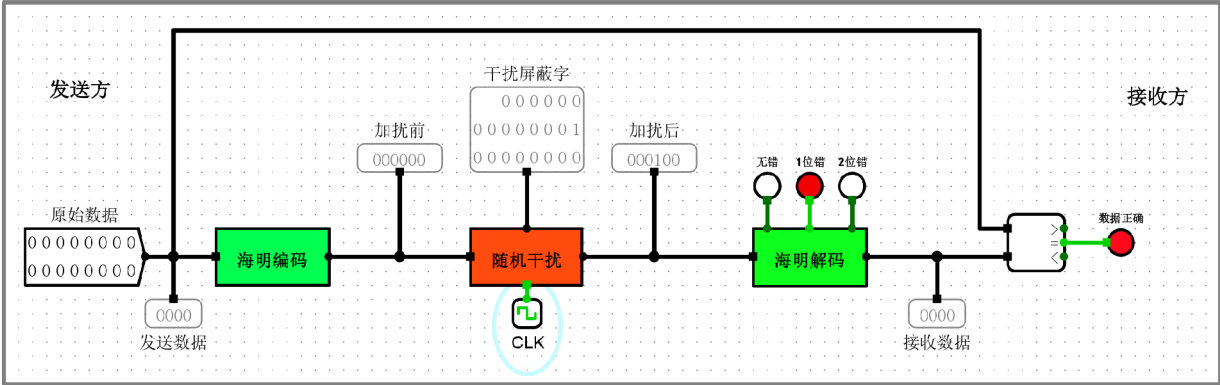
- **电路功能：**在接收端接收 17 位数据，判断传输过程中是否发生奇数位错误。
- **实现原理：**检错电路同样利用了异或门阵列，但其处理对象是包含校验位在内的完整 17 位数据。

其逻辑规则如下：

- **无错状态：**由于编码端保证了整体“1”的个数为偶数，检错电路对 17 位数据进行异或运算后，结果应为 **0**。电路通过指示灯显示“无错”。
- **检错状态：**若传输过程中发生单比特翻转，数据的奇偶性发生改变，异或运算输出变为 **1**。电路捕捉到该逻辑电平，驱动报警指示灯亮起，从而实现对数据传输错误的实时检测。

(4)16位扩展海明码传输测试实验

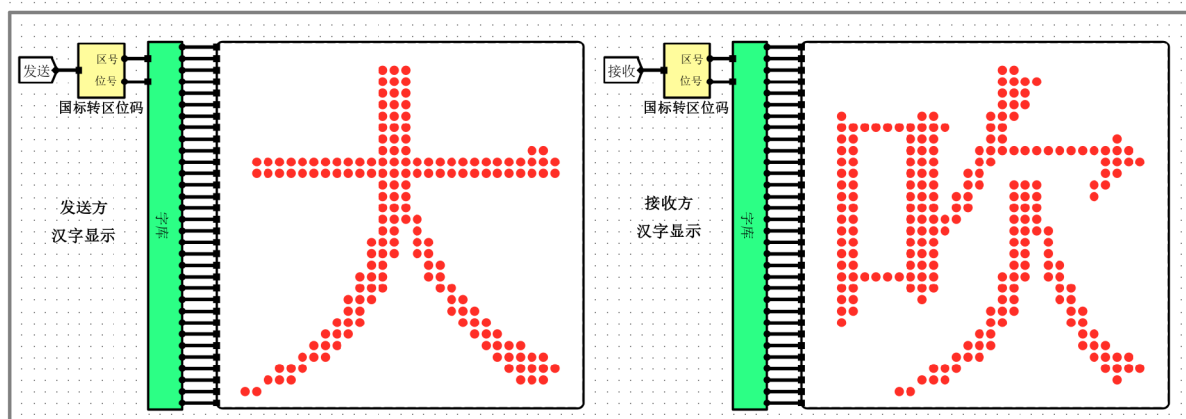
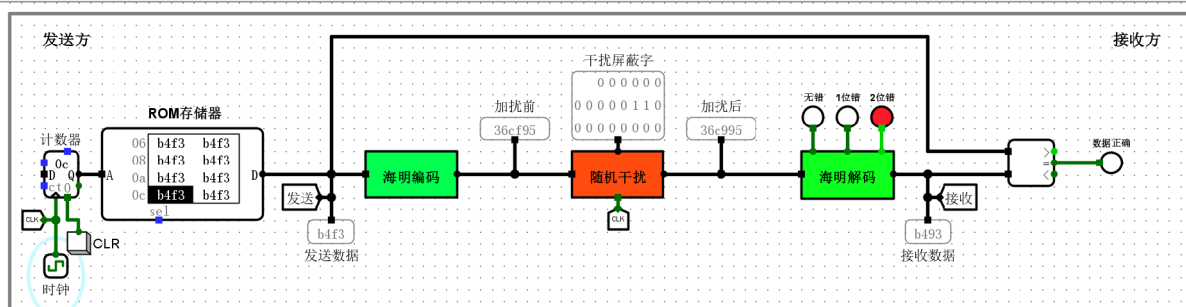
实验结果



电路功能：测试扩展海明编解码电路的正确性。点击“CLK”按钮2次，观看4个指示灯，有以下4种情况：

- 1、无错：无错指示灯红色，数据正确指示灯红色，干扰屏蔽字=0，接收数据等于发送数据
- 2、1位错：1位错指示灯红色，数据正确指示灯红色，干扰屏蔽字中有1个1，接收数据等于发送数据（可以纠正1位错）
- 3、2位错：2位错指示灯红色，数据正确指示灯白色，干扰屏蔽字中有2个1，接收数据不等于发送数据（不能纠正2位错）
- 4、2位错：2位错指示灯红色，数据正确指示灯红色，干扰屏蔽字中有2个1，接收数据等于发送数据（此时可以纠正错误）

16位扩展海明码传输测试实验1



电路功能：测试扩展海明编码电路正确性。先点击“CLR”按钮，使计数器清零，从头开始显示汉字；然后每点击“时钟”按钮2次，取出1个汉字；将有4种情况：

- 1、无错：无错指示灯红色，干扰屏蔽字=0，接收数据等于发送数据，接收方与发送方显示相同的汉字。
- 2、1位错：1位错指示灯红色，干扰屏蔽字中有1个1，接收数据等于发送数据，接收方与发送方显示相同的汉字（可以纠正1位错）。
- 3、2位错：2位错指示灯红色，干扰屏蔽字中有2个1，接收数据不等于发送数据，接收方与发送方显示不同的汉字（不能纠正2位错）。
- 4、2位错：2位错指示灯红色，干扰屏蔽字中有2个1（并且最高位为1），接收数据等于发送数据，接收方与发送方显示相同的汉字（此时可以纠正错误）。

16位扩展海明码传输测试实验2

实验分析

偶校验编码电路分析

分组校验生成 ($P_1 \sim P_5$):

- 电路利用 5 棵异或树，分别根据二进制位权规则（如我们之前讨论的 P_1 管辖位权最低位为 1 的数据位）计算出 5 个海明校验位。
- 这 5 位负责后续的**单比特错误定位**。

总偶校验位生成 (P_6):

- 这是一个关键的扩展点。电路中有一个覆盖面积最大的异或阵列，它将 **16 位原始数据 + 5 位已生成的校验位** 全部作为输入。
- 逻辑核心**: $P_6 = D_0 \oplus D_1 \oplus \dots \oplus D_{15} \oplus P_1 \oplus \dots \oplus P_5$ 。
- 目的**: 确保最终输出的 22 位编码中，1 的总个数必须为偶数。这为区分“单比特错”和“双比特错”提供了全局判据。

偶校验检错电路分析

校验子 (Syndrome) 计算:

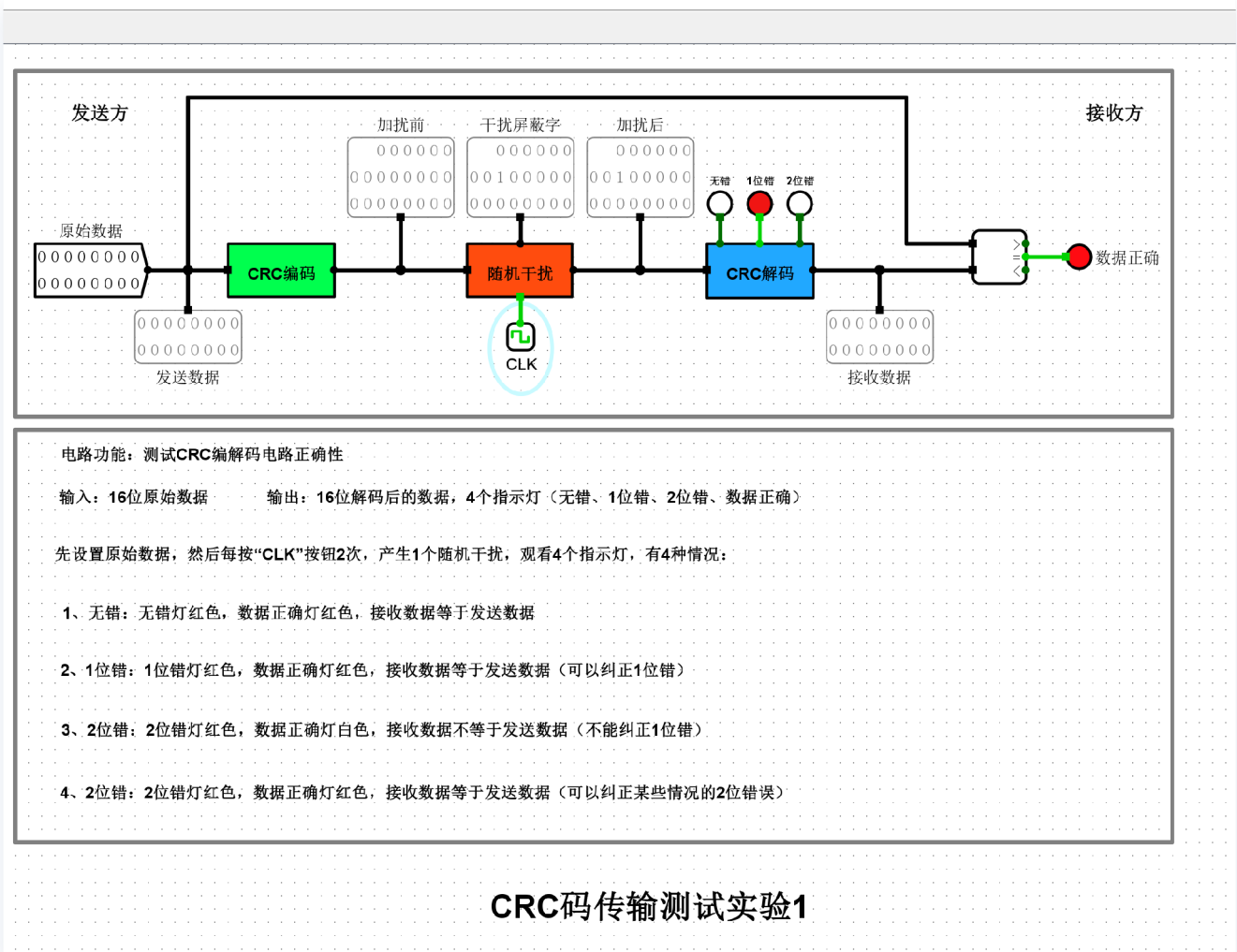
- 解码器重新计算 $P_1 \sim P_5$ 的奇偶性, 生成 5 位校验子 $G_1 \sim G_5$ 。
- 同时, 电路会计算包含 P_6 在内的所有 22 位的总奇偶性, 得到一个全局校验子 G_6 。

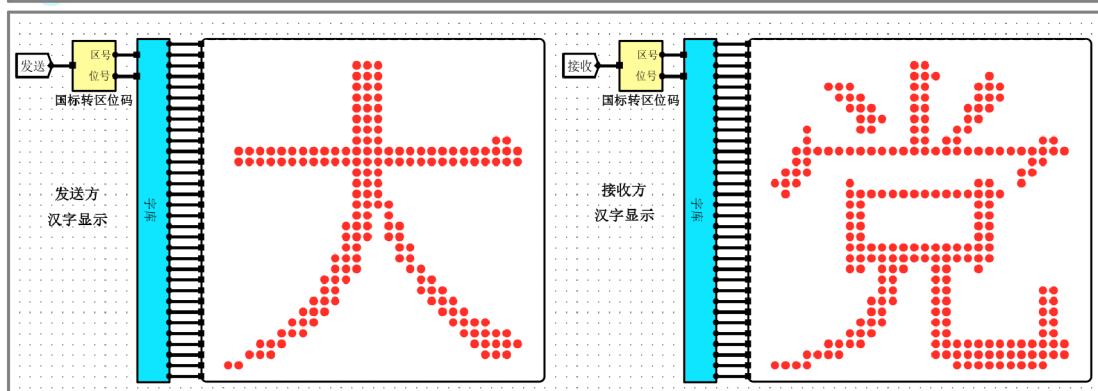
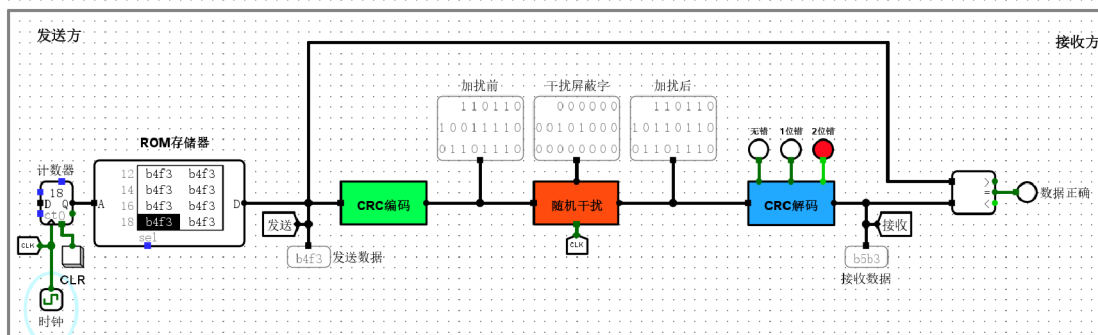
逻辑判定分支:

1. **无错**: 如果 $G_1 \sim G_5$ 全为 0, 且 $G_6 = 0$, 说明传输完美无缺。
2. **单比特错误 (可纠正)**: 如果 $G_1 \sim G_5$ 不全为 0 (指向了错误坐标), 且 $G_6 = 1$ (全局校验发现奇偶性确实变了), 则电路确认这是单错, 启动译码器纠错。
3. **双比特错误 (仅检错)**: 如果 $G_1 \sim G_5$ 不全为 0, 但 $G_6 = 0$ (说明发生了两位错, 奇偶性相互抵消, 全局校验没变), 电路判定为**双错**。此时电路会点亮“2位错”指示灯, 并封锁纠错输出, 防止把正确位改错。

(5)循环冗余校验码 (并行)

实验结果





电路功能：测试CRC编解码电路正确性

输入：16位汉字国标码（存放在ROM存储器中） 输出：16位解码后的数据（汉字国标码），4个指示灯（无错、1位错、2位错、数据正确），接收方与发送方显示的汉字

先按“CLR”按钮，使计数器清零；然后每按“时钟”按钮2次，输出1个汉字的国标码，并产生1个随机干扰；观看4个指示灯和发送方与接收方显示的汉字，有4种情况：

- 1、无错：无错灯红色，数据正确灯红色，接收数据等于发送数据
- 2、1位错：1位错灯红色，数据正确灯红色，接收数据等于发送数据（可以纠正1位错）
- 3、2位错：2位错灯红色，数据正确灯白色，接收数据不等于发送数据（不能纠正1位错）
- 4、2位错：2位错灯红色，数据正确灯红色，接收数据等于发送数据（可以纠正某些情况的2位错误）

CRC码传输测试实验2

实验分析

CRC 并行编码电路分析

电路功能：

该电路通过并行逻辑将 16 位原始数据一次性映射为 22 位 CRC 编码（16 位数据 + 6 位校验位），生成多项式固定为 $G(X) = 100101$ 。

内部逻辑拆解：

• 输入与移位预处理：

16 位数据 $D_1 \sim D_{16}$ 同时输入。在数学上，这相当于将原始数据多项式左移了 6 位（因为校验位是 6 位）。

• 异或阵列：

实际上是**多项式模2除法的硬件固化**。

- 每一路生成的校验位 $P_1 \sim P_5$ 都连接了一组特定的 D 信号。这些连线关系是由生成多项式 100101 决定的。
- **并行特性**：它不像串行电路那样需要时钟触发，而是数据一旦进入，经过门电路延迟后，校验位就直接在输出端产生了。
- **总偶校验集成 (P_6)**：

右侧那个巨大的树状异或阵列，它汇聚了所有的 $P_1 \sim P_5$ 以及部分数据信号。这说明该 CRC 设计不仅做了多项式校验，还集成了一个**全局奇偶校验 (P_6)**，用于增强对多比特错误的检测鲁棒性。

CRC 并行解码电路分析

电路功能：

接收 22 位 CRC 编码，判定数据传输是否正确，并具备“检测并纠正单比特错误”的功能。

内部逻辑拆解：

- **伴随式计算：**

解码电路同样包含一组与编码端镜像的异或阵列。它将收到的 22 位数据重新进行“模2除法”运算。如果传输无误，算出的余数应全为 0。

- **错误类型判定：**

有三个关键的指示灯：“**无错误**”、“**1位错**”、“**2位错**”。

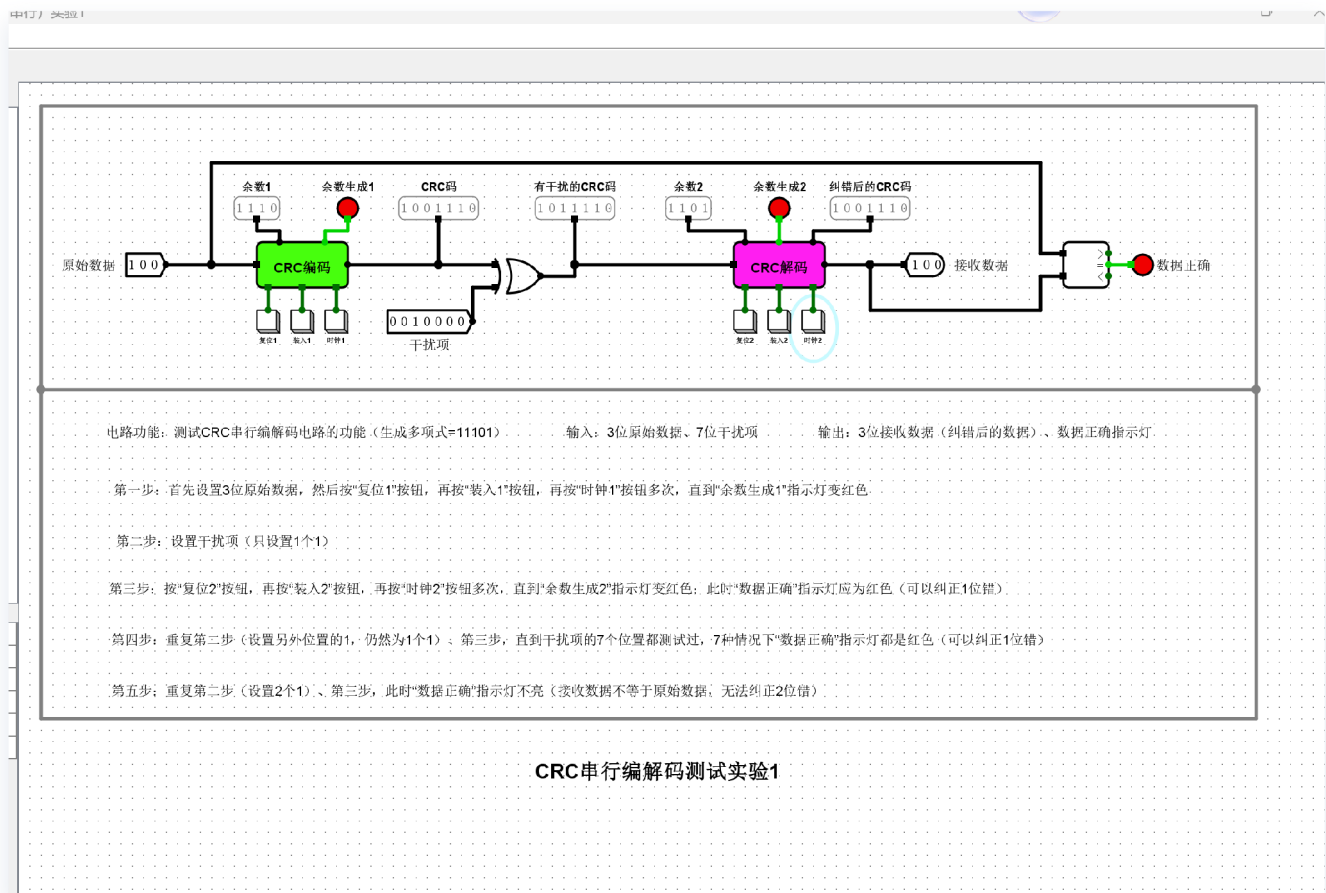
- **无错误**：当所有异或运算结果均为 0 时，与非门触发，点亮“无错误”灯。
- **1位错与自动纠正**：如果伴随式不为 0，且符合特定的单比特错误特征（通常结合了全局校验位 P_6 的状态），电路会判定为 1 位错。
- **2位错检测**：如果局部校验显示出错，但总奇偶性 (P_6) 未发生预期改变，电路判定发生双比特错误。此时，“2位错”灯亮起，通知系统数据不可信。
- **纠错反馈通路：**

图中下方那一排 **MUX** 和 **XOR 门** 是纠错的核心。

- 译码器根据算出的余数定位出错的比特。
- 对应的 MUX 会切换通路，通过异或门将错误位取反，最后将“纠错后的数据”输出到最右侧的信号区域。

(6)循环冗余校验码（串行）1

实验结果



实验分析

解码

解码器在接收端负责对接收到的完整码流进行校验。

- **校验逻辑：**接收到的数据（包含校验码）被再次输入到相同的 LFSR 电路中进行模2除法运算。若传输过程中无错误发生，则除法后的余数应为0。
- **错误检测：**电路末端集成了一组全零检测逻辑，通过多输入与门或或非门阵列监控所有触发器的输出状态。当且仅当所有触发器输出均为0时，输出逻辑高电平，指示“无错误”；否则，电路输出报警信号，提示数据在传输中受损。

编码

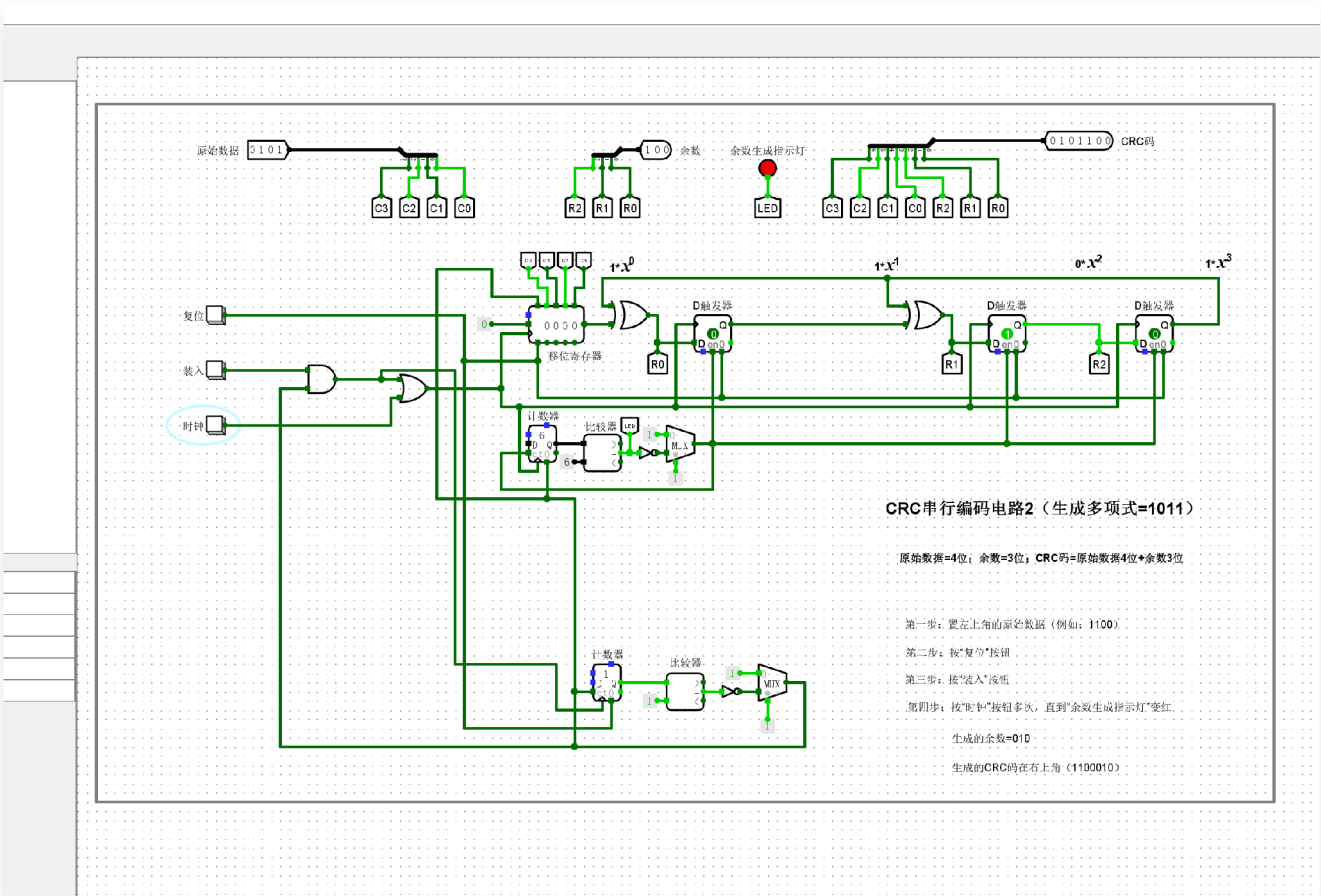
编码器的核心功能是计算校验冗余码。电路采用串行输入方式，数据在时钟脉冲驱动下逐位移入移位寄存器。

- **数据处理：**在数据输入阶段，LFSR 结合反馈逻辑进行模2除法运算。

- **余数生成：**原始数据输入完毕后，移位寄存器中保留的最终状态即为 CRC 余数。该余数将被附加在原始数据之后发送，形成完整的传输帧。

(7)循环冗余校验码（串行）2

实验结果



实验分析

解码

解码电路通过计算接收到的 CRC 码与生成多项式的模 2 余数，判定数据传输过程中是否发生误码。

- **核心构成：**
 - **余数计算单元：**与编码逻辑相似，通过相同的除法电路对接收到的完整码字进行模 2 除法。
 - **校验判决逻辑：**位于电路底部的比较器阵列。
 - **状态指示：**若余数为 0，则 LED 灯不亮（表示校验通过，无错误）；若余数非 0，则 LED 亮起（表示数据在传输中出现错误）。
- **解码原理：**
 - 若接收的码字（原始数据 + 校验位）能被生成多项式整除（余数为 0），则认为传输正确。

- 解码电路在完成运算后，通过比较器对每一位进行校验，确保数据完整性。

编码

编码电路的主要任务是将原始数据帧与生成多项式进行模 2 除法，计算出校验位。

- 核心构成：

- 移位寄存器：作为数据输入的缓存，按时钟脉冲进行移位。
- 除法逻辑：通过异或门阵列实现模 2 除法。当移位寄存器最高位为 1 时，反馈路径将寄存器内容与生成多项式（ 1011 ）进行异或运算。
- 计数器与控制逻辑：用于控制移位过程。当计数器达到设定值（原始数据位数）时，表示除法完成，此时移位寄存器中的剩余数据即为 CRC 校验位。

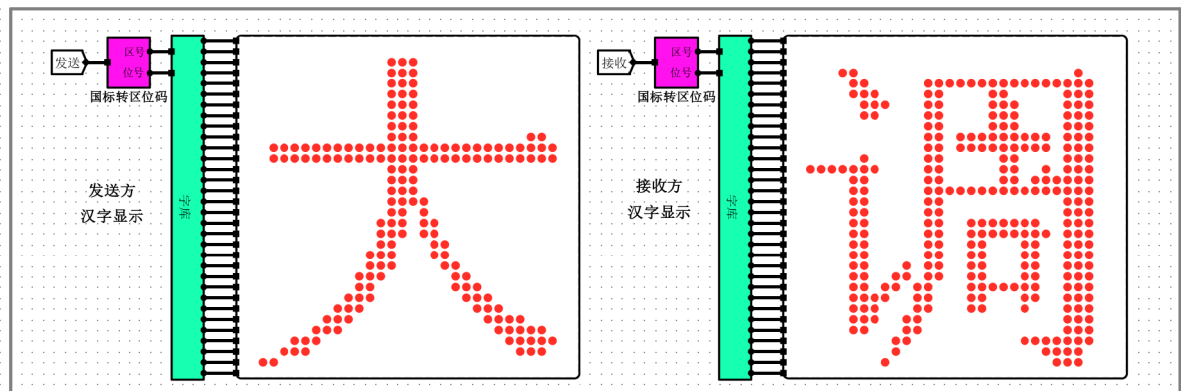
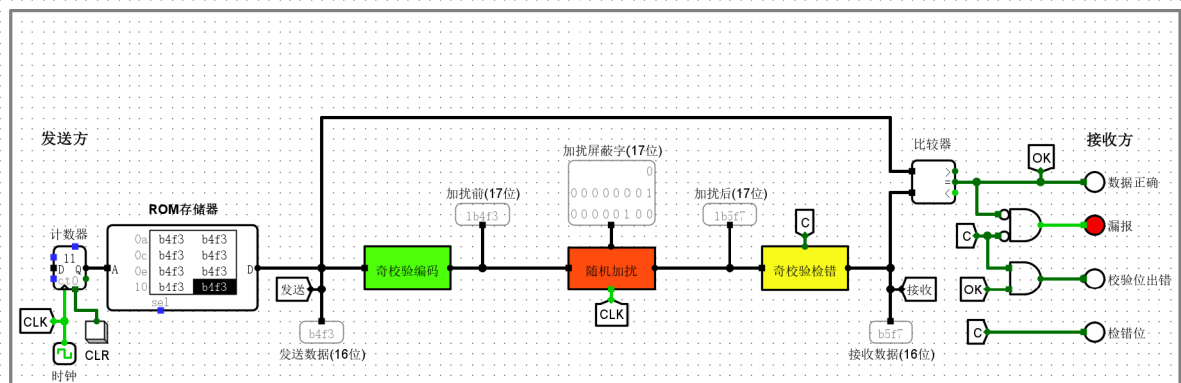
- 编码步骤：

1. 复位：将除法电路清零。
2. 装入：将原始数据载入移位寄存器。
3. 计算：通过时钟驱动进行循环移位与异或运算，直到余数生成。
4. 结果：将生成的校验位附加到原始数据后，形成最终的 CRC 码。

3.2设计实验

(1)16位奇校验传输测试实验（16位奇校验传输测试实验.circ）

实验结果



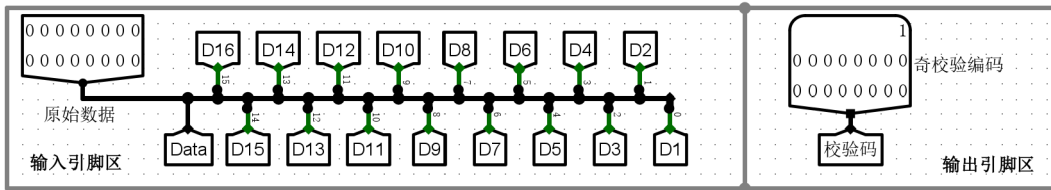
电路功能：按“CLR”将计数器清零；按“时钟”2次，计数器加1，ROM存储器送出1个汉字的国标码。会有4种情况出现：

- 1、数据正确=1（红灯）、检错位=0（白灯），表示没有错误（干扰屏蔽=0），接收方与发送方显示的汉字相同
- 2、数据正确=0（白灯）、检错位=0（白灯），误报=1（红灯），表示误报（没有检出错，但是接收到的数据却出错了），接收方与发送方显示的汉字不同
- 3、数据正确=0（白灯）、检错位=1（红灯），表示检出错误，接收方与发送方显示的汉字不同
- 4、数据正确=1（红灯）、检错位=1（红灯），表示校验位出错、数据位没有出错，接收方与发送方显示的汉字相同

16位奇校验传输测试实验

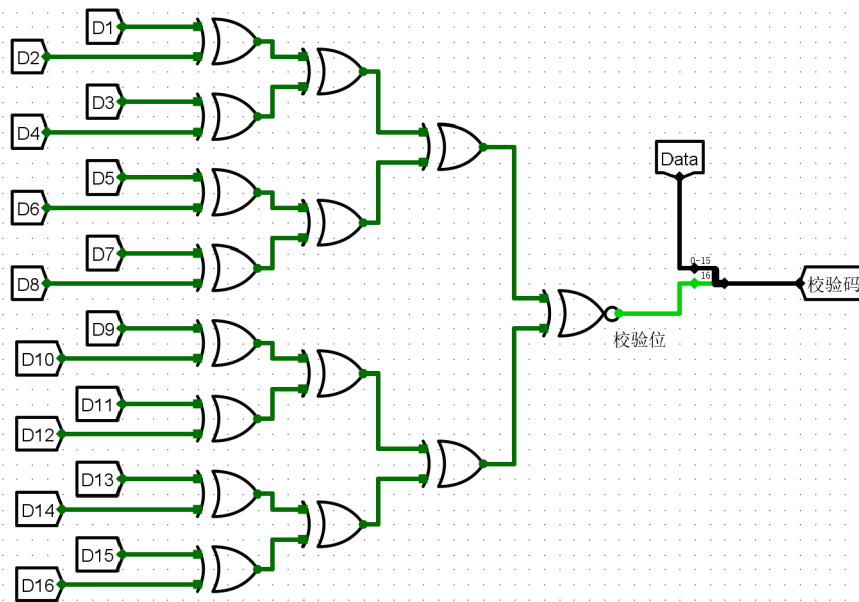
实验分析

编码电路分析



电路功能：实现16位数据位的奇校验编码

输入：16位数据；输出：17位校验码（数据位+校验位）



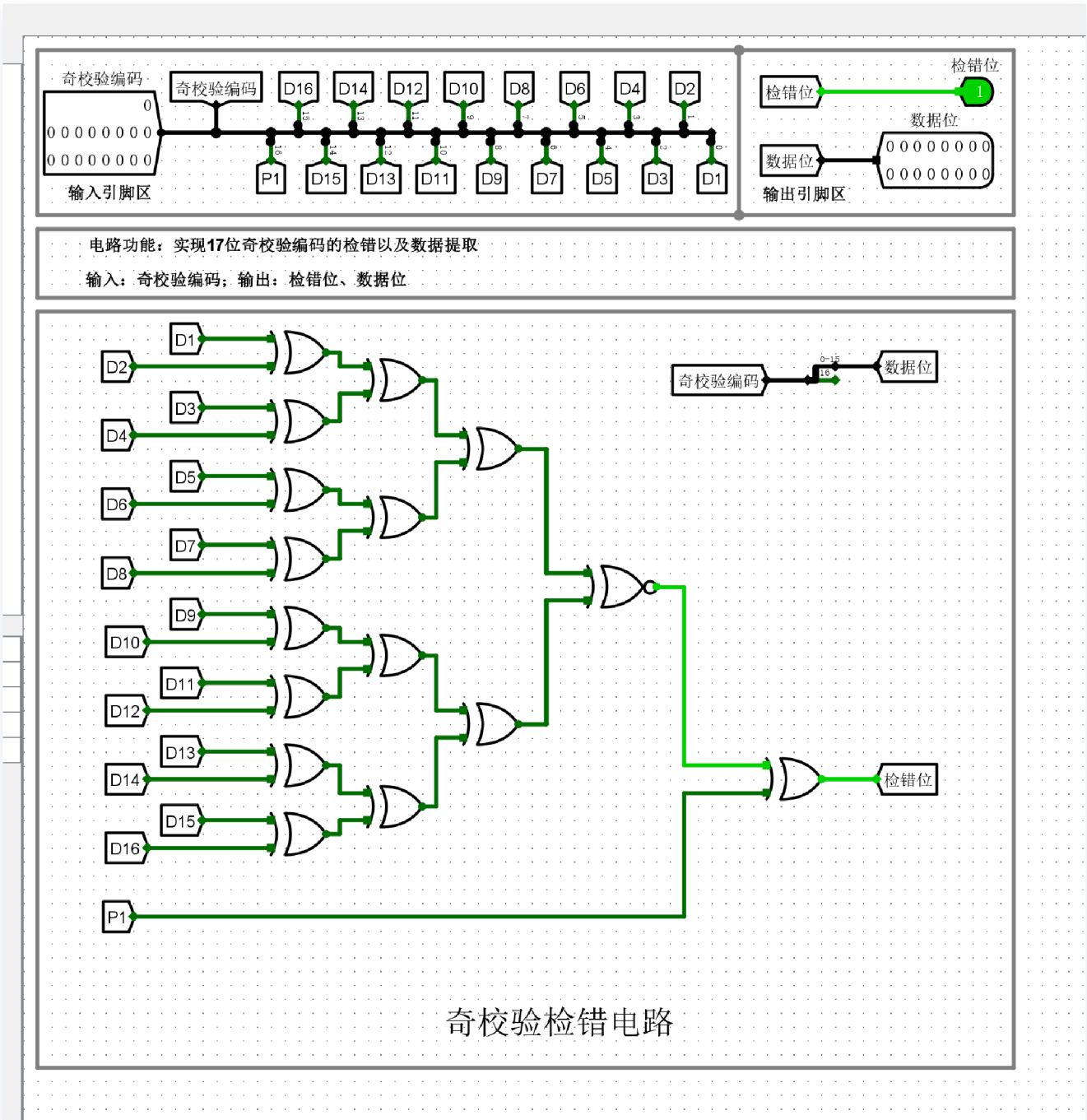
奇校验编码电路

编码器的核心功能是根据输入的 16 位数据产生校验位。

- **逻辑实现：** 原电路采用多级 XOR 树结构，其输出为 $P = D_{16} \oplus D_{15} \oplus \cdots \oplus D_1$ 。将其改为奇校验后，校验位 P' 应满足 $P' = \overline{D_{16} \oplus D_{15} \oplus \cdots \oplus D_1}$ 。
- **优化更改：** 将末端逻辑运算由 XOR 门替换为 XNOR 门。

XNOR 的逻辑定义为：仅当输入全为 0 或输入中 1 的个数为偶数时输出 1。这确保了当 16 位数据中 1 的个数为偶数时，输出 1，使数据位与校验位中 1 的总数为奇数，从而实现奇校验编码。

验错电路分析

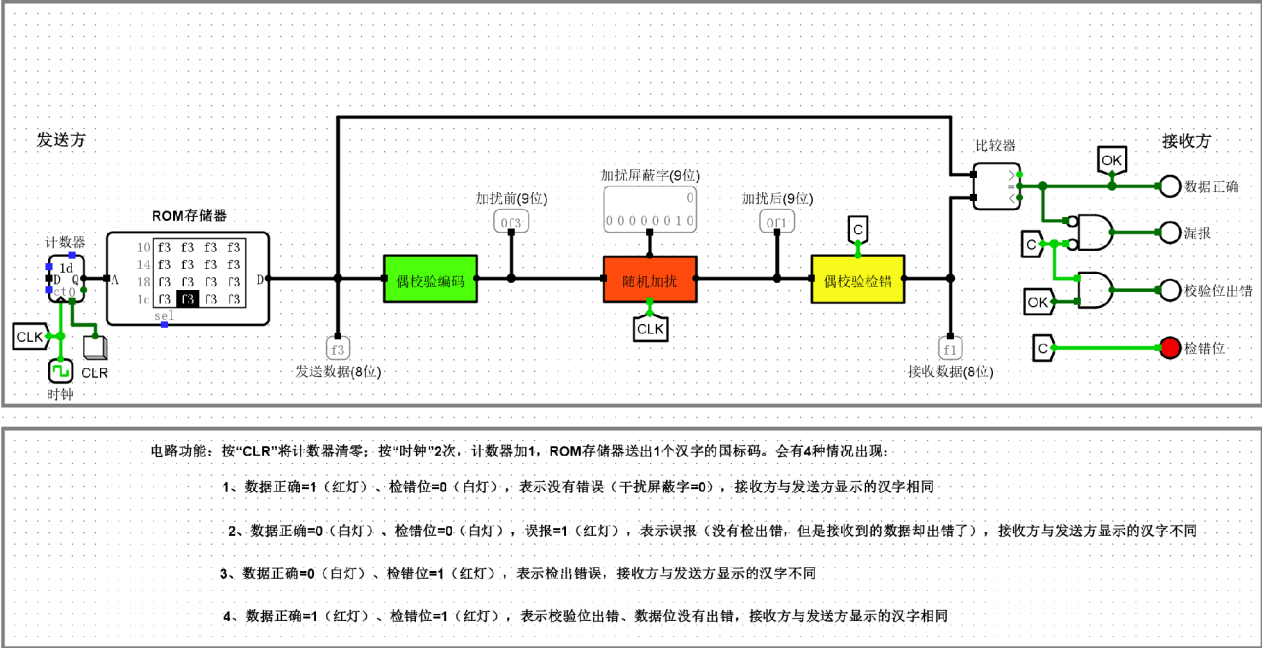


验错电路的任务是检查接收到的 17 位数据（16 位数据 + 1 位校验位）是否符合奇校验规则。

- **逻辑实现：** 同样采用 XOR 树结构，将 17 位输入数据进行异或求和。
- **校验规则：** 对于奇校验，若无传输错误，则 17 位数据的异或和取反应始终为 0。
- **更改影响：** 验错端的 XOR 树输出若为 0，则判定数据正确；若输出为 1，则判定数据在传输过程中出现了单比特错误。

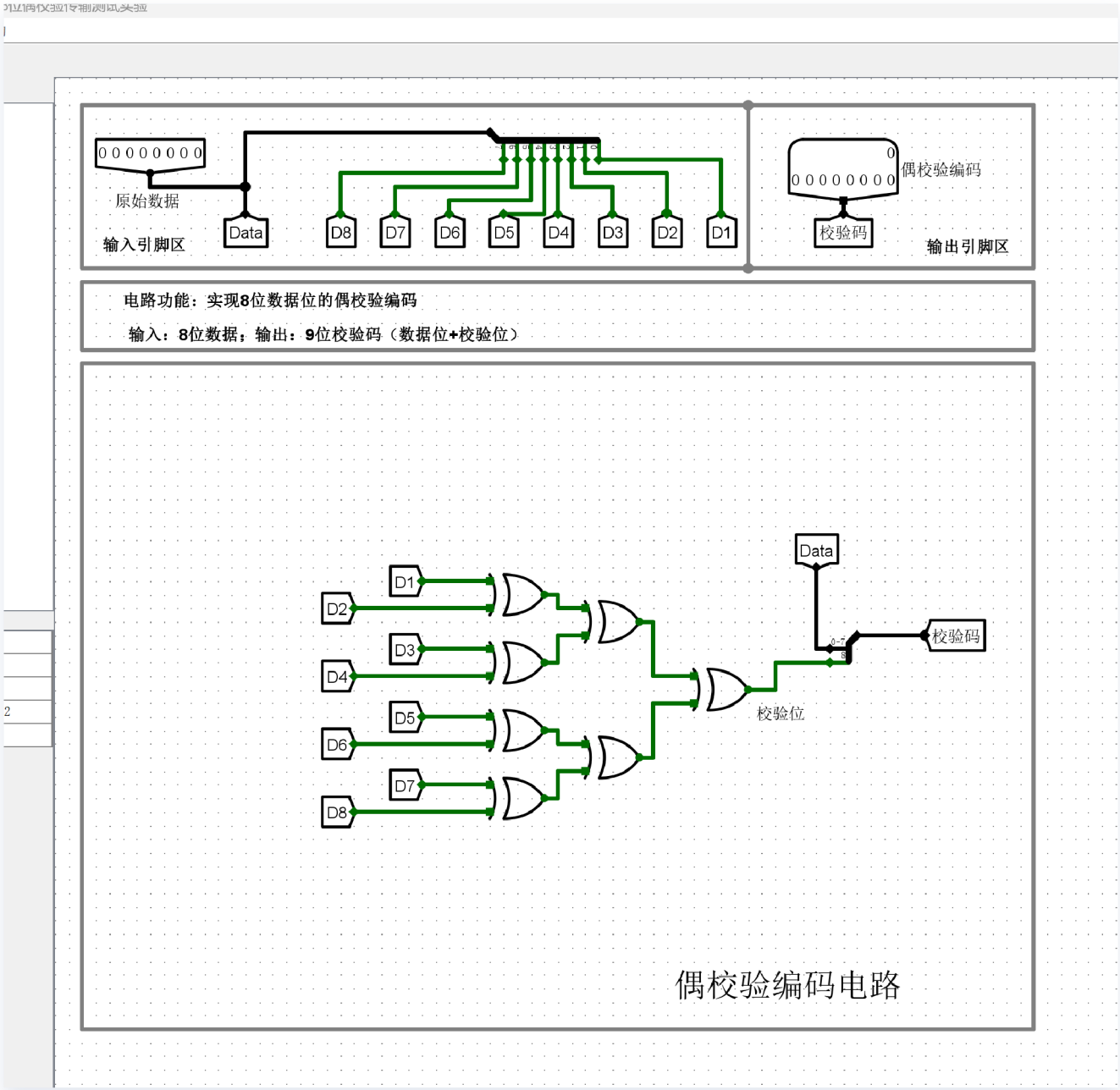
(2)8位偶校验传输测试实验（8位偶校验传输测试实验.circ）

实验结果



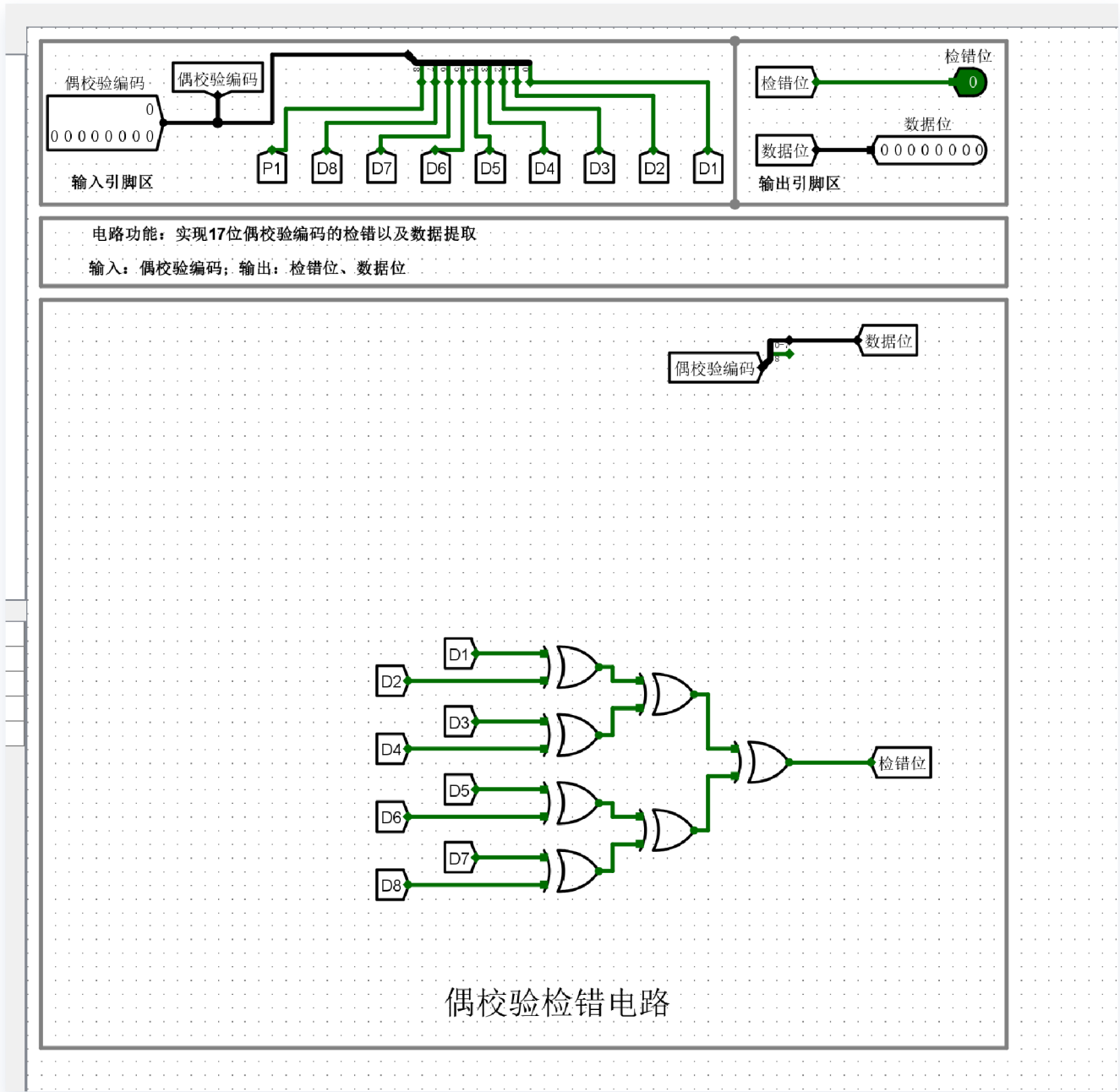
8位偶校验传输测试实验

编码电路分析



- 编码器的功能是根据 8 位数据生成 1 位校验码，使数据位与校验位中“1”的总数为偶数。
- **逻辑实现：** 将原 16 输入的 XOR 树结构裁剪为 8 输入。电路通过 3 级异或门级联，实现了 $P = D_1 \oplus D_2 \oplus \dots \oplus D_8$ 的模 2 加法运算。
 - **修改说明：** 移除了 D_9 至 D_{16} 的输入引脚及对应的多级 XOR 门逻辑，将 XOR 树的输入规模从 16 位缩减至 8 位。

验错电路分析

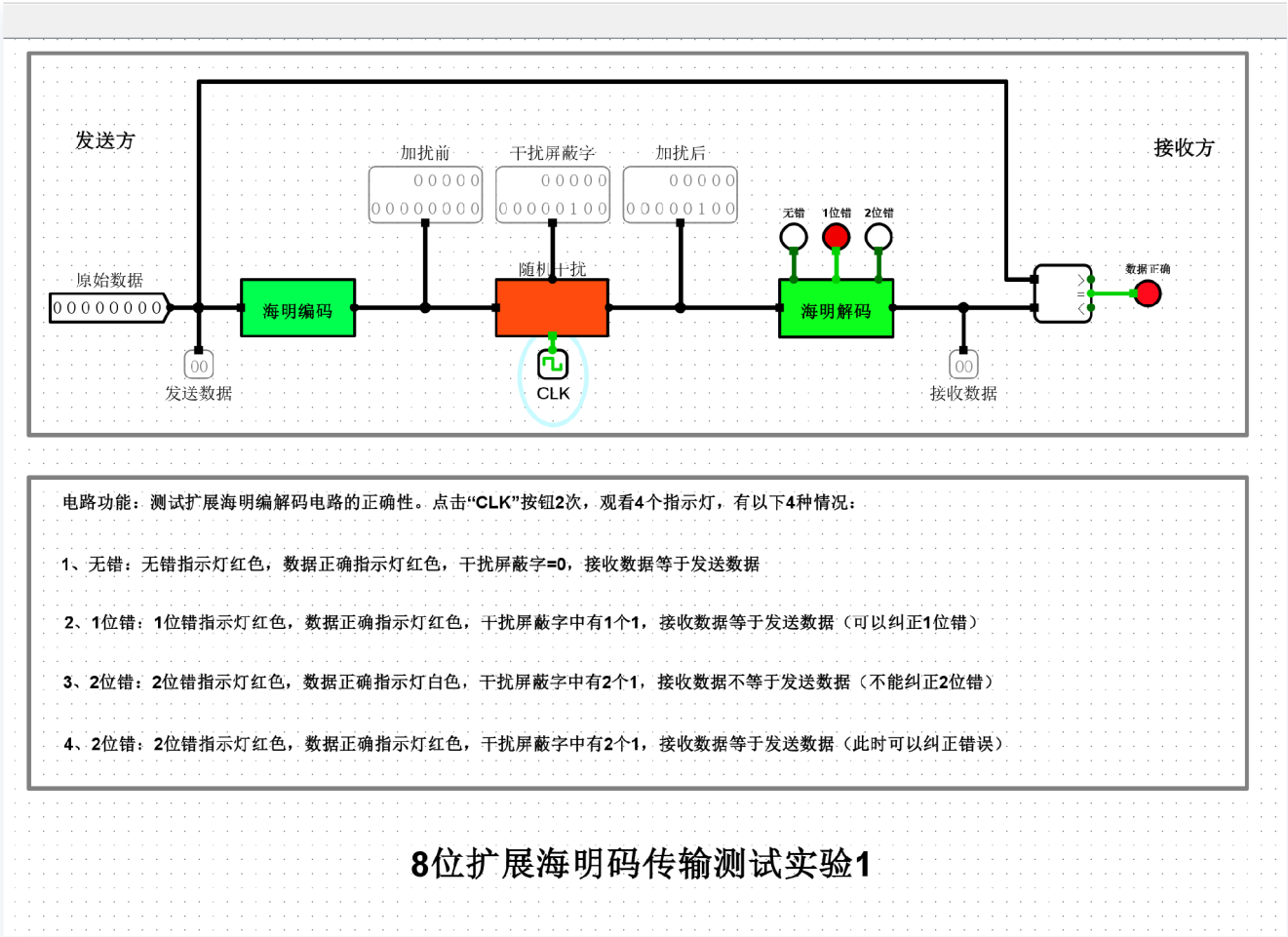


验错电路的任务是检查包含校验位在内的 9 位数据是否符合偶校验规则。

- **逻辑实现：** 将原 17 位输入（16 位数据+1 位校验码）的 XOR 树结构裁剪为 9 位输入。
- **校验规则：** 电路对 8 位数据及 1 位校验码进行异或求和。若传输无误，总的异或结果应为 0（即“1”的总数为偶数）。
- **修改说明：** 同样剔除了多余的输入分支，调整后的 XOR 树能够正确处理 9 位数据流。当异或结果输出为 1 时，即判定为传输过程中发生了单比特错误。

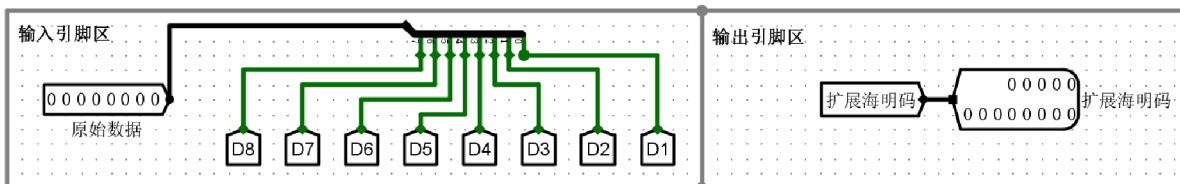
(3)8位扩展海明码传输测试实验（8位扩展海明码传输测试实验.circ）

实验结果



实验分析

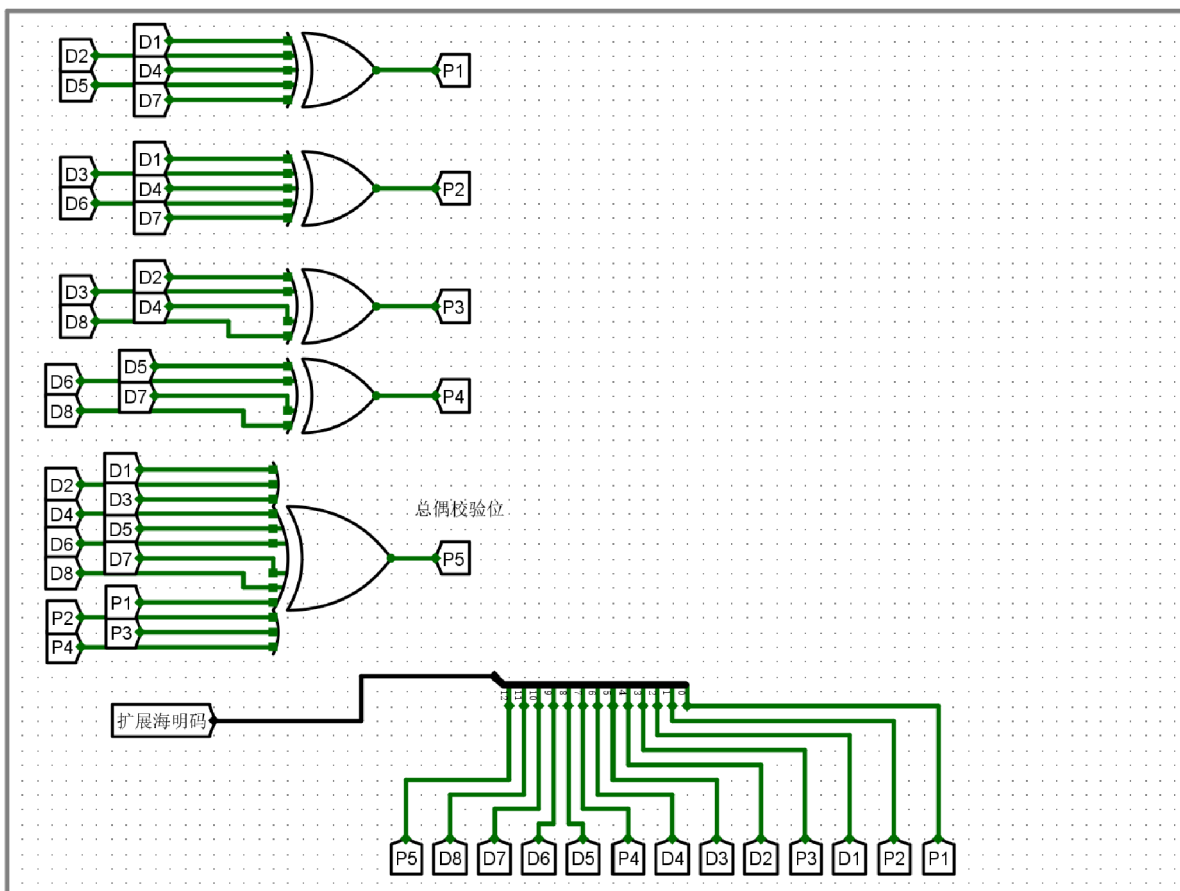
编码电路分析



电路功能：扩展海明码编码电路

输入：8位原始数据

输出：13位海明码（8位数据位+4位校验位+1位总偶校验位）

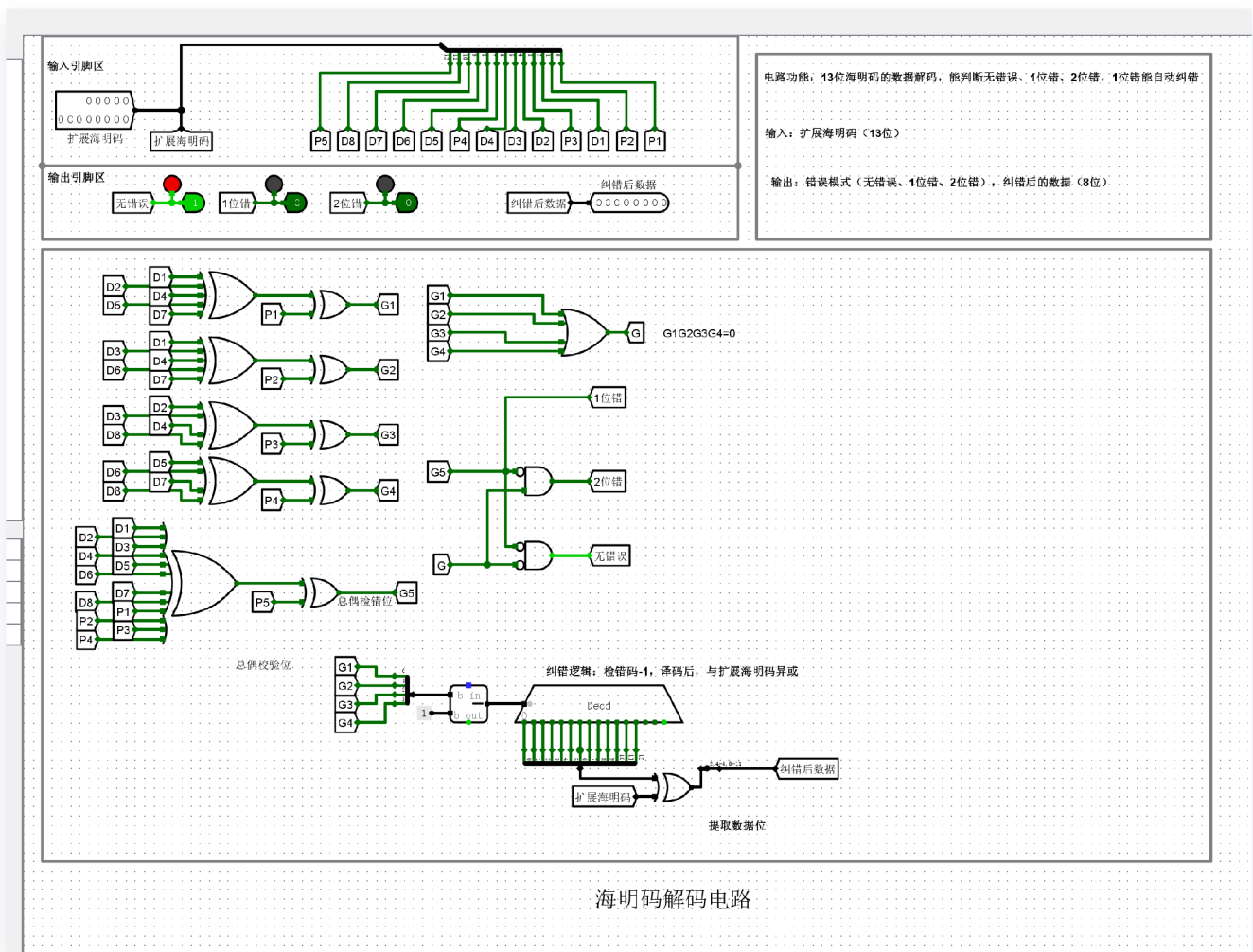


扩展海明码编码电路

编码器的任务是将 8 位原始数据映射到 13 位扩展海明码序列中。

- **规模缩减：** 删除了原 16 位电路中的 D_9 至 D_{16} 输入引脚，将校验位生成逻辑从 5 组 ($P_1 \sim P_5$) 减少为 4 组 ($P_1 \sim P_4$)。
- **校验逻辑调整：** 根据海明码位置分配规律，数据位 $D_1 \sim D_8$ 分别占据总序列的第 3, 5, 6, 7, 9, 10, 11, 12 位。通过多输入 XOR 门计算各小组异或和，例如 P_1 负责校验位位置二进制编号末位为 1 的小组，其输入缩减为 D_1, D_2, D_4, D_5, D_7 。
- **总偶校验 (P_5)：** 最后一组增加了一个大输入的 XOR 门，对前 12 位（8 位数据+4 位校验位）进行全局异或，生成第 13 位总偶校验位，以实现 SEC-DED（单错纠正-双错检测）功能。

解码电路分析

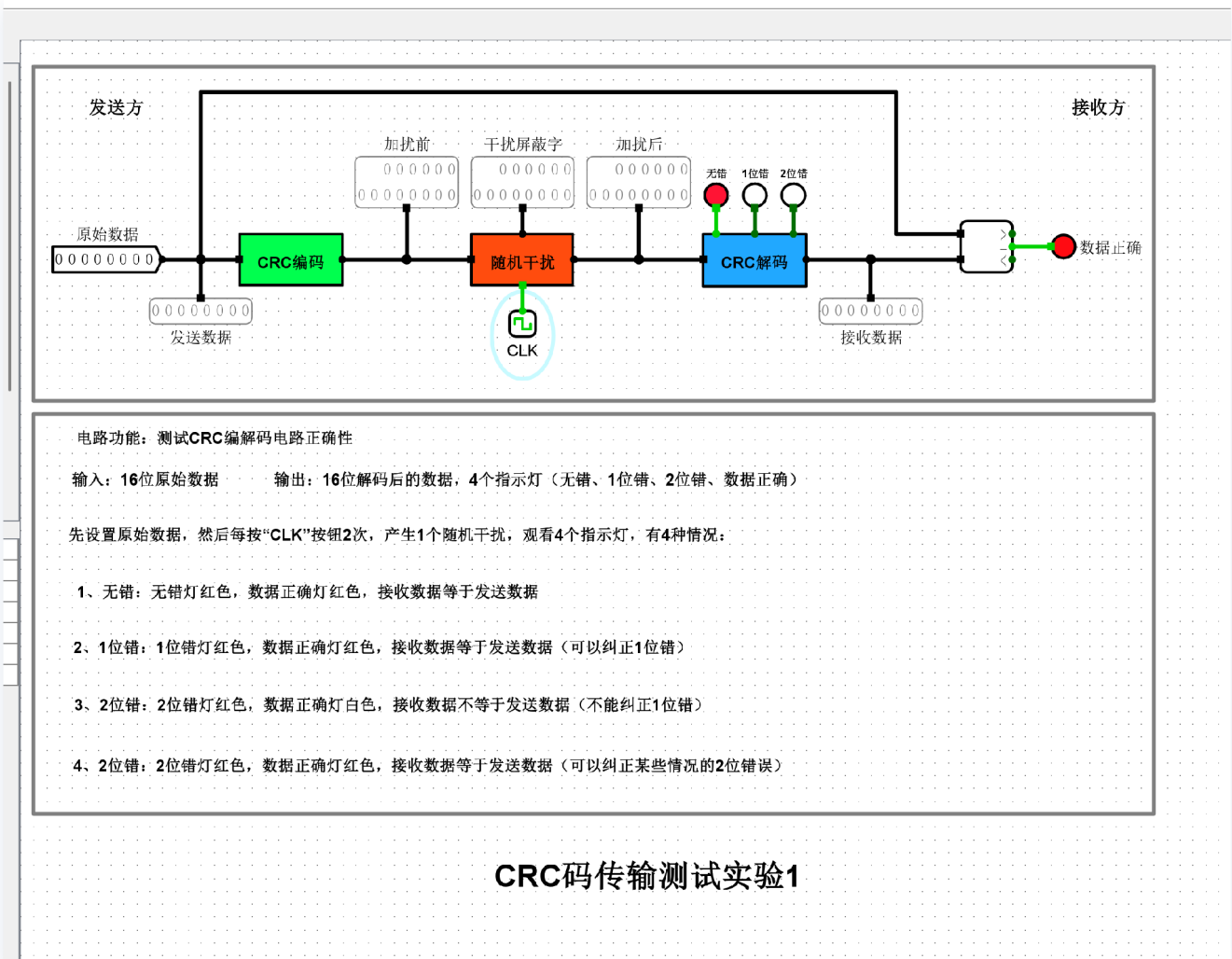


解码器用于检测并纠正传输中产生的错误。

- **指指字（Syndrome）生成：** 将接收到的数据按编码规则重新进行 XOR 运算，生成 4 位指指字 $G = G_4G_3G_2G_1$ 。
- **纠错逻辑修改：** * 将 5 位译码器更换为 **4 位译码器**。指指字 G 的值直接对应错误发生的位置（1-12）。
 - **数据提取映射：** 纠错后的 12 位序列中，仅通过 XOR 门对处于数据位位置（3, 5, 6, 7, 9, 10, 11, 12）的信号进行翻转纠正，并剔除 1, 2, 4, 8 处的校验位，重新拼合为 8 位原始数据。
- **错误类型判定：**
 - 引入全局校验位检测结果 G_5 。
 - 若指指字 $G \neq 0$ 且 $G_5 = 1$ ，判定为 **1 位错**，指示灯亮并自动纠错。
 - 若指指字 $G \neq 0$ 且 $G_5 = 0$ ，判定为 **2 位错**，仅指示错误，不进行纠正。
 - 若 $G = 0$ 且 $G_5 = 0$ ，判定为 **无错误**。

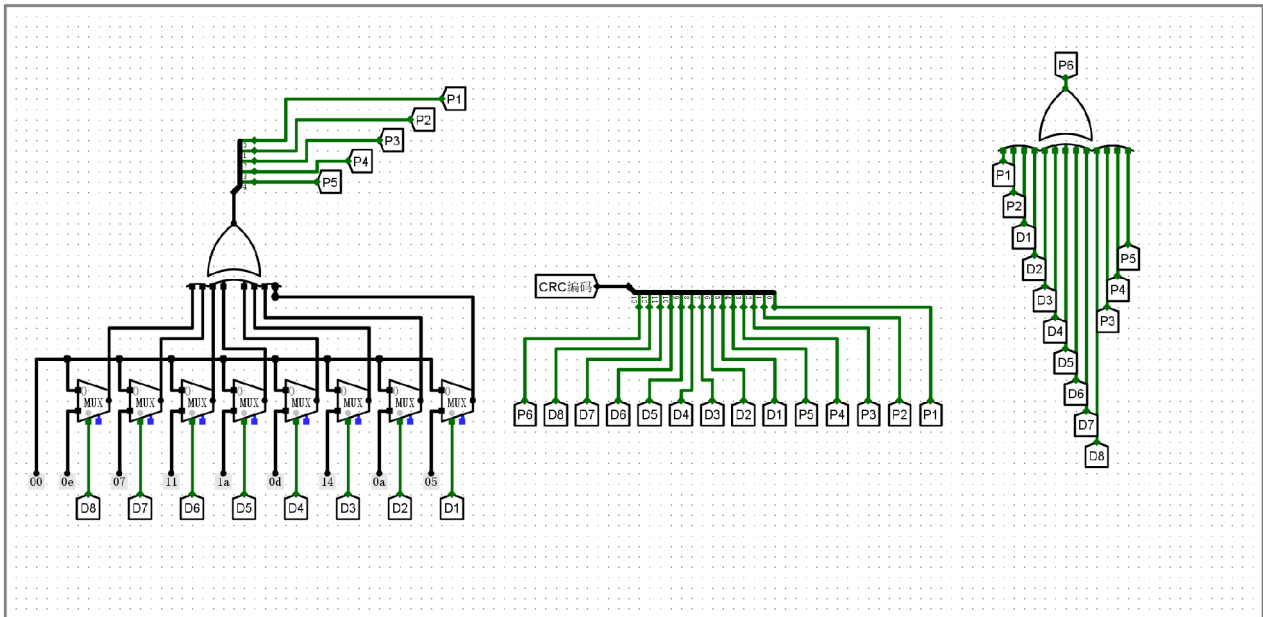
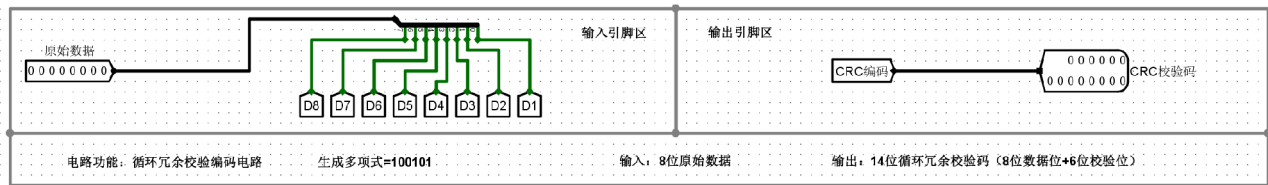
3.3挑战性实验（8位循环冗余校验码（并行）实验.circ）

实验结果



实验分析

编码电路分析

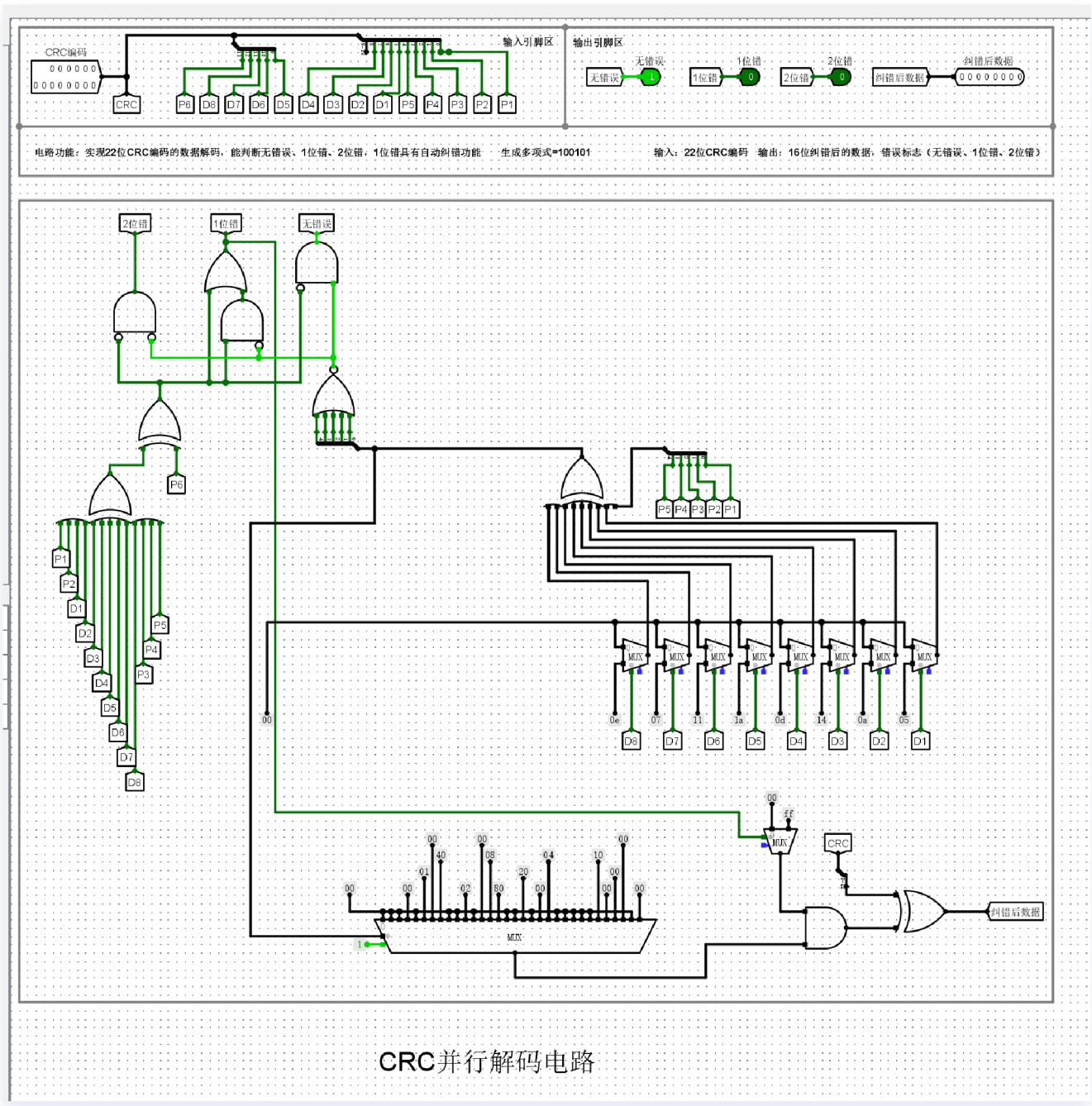


CRC并行编码电路

在8位设计中，我采用了预计算矩阵法：

- 参数重构：**通过计算，得到了8位数据输入下每一位的对应十六进制权重。例如， D_8 对应权重 `0x0e`， D_7 对应 `0x07` 等。这些值被映射到电路的MUX阵列中，作为该数据位是否参与异或运算的逻辑控制基准。
- 电路结构：**编码电路移除了原16位电路中多余的输入逻辑，保留了8个MUX组。每个MUX根据输入 D_i 的状态，选择输出对应的权重值，最终通过一个5路并行XOR门树（XOR Tree）实现模2加法，直接得出5位校验码。

解码电路分析



译码电路采用了“伴随式（Syndrome）校验”逻辑：

- **计算伴随式**：译码器接收传输后的数据（8 位数据 + 5 位校验码），利用同样的并行逻辑计算出接收端的伴随式 S 。
- **纠错逻辑**：
 - **无错误判定**：若 $S = 0$ ，则表明数据传输正确。
 - **1 位纠错**：若 S 的值与某一位产生的错误模式匹配（即错误向量表），则触发纠错逻辑，通过 MUX 将该位反转。
 - **2 位检错**：利用多项式检测能力，识别出无法纠正的 2 位及以上错误，并发出错误标志位。